

Two Pointers TECHNIQUE

- Q Given N distinct elements in a sorted array,
check if there is pair st $a[i] + a[j] = k$

$$i \neq j$$

[1, 3, 4, 6, 7, 10, 13, 14, 20]

$$k = 15$$

Unsorted
Array

- ① Brute Force
(Two loops)

time

$$O(N^2)$$

space

$$O(1)$$

- ② Sorting
+ Binary Search

$$O(N \log N)$$

$$O(N)$$

(Merge sort)

- ③ Sorting +
Two Pointers

$$O(N \log N)$$

$$O(N)$$

Merge Sort

- ~~④~~ Hashset
for unsorted
array

$$O(N)$$

$$O(N)$$

[3, 5, 2, 1, 8]

$$k = 10$$

1, 2, 3, 5, 8
↑ ↑ ↑

$$(1, 9)$$

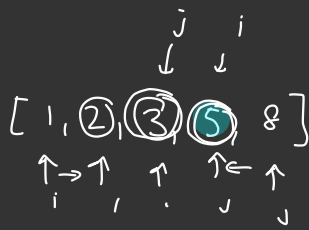
$$(2, 8) \checkmark$$

$$1 + 8 = 9 < 10$$

$$2 + 8 = 10 == 10$$

$$(2, 8)$$

Sorted
Array



$K=8$

$O(N)$ with $O(1)$ space

Two
Pointer
app.

$$a(i) + a(j) = 9 > 8 \quad j--$$

$$a(i) + a(j) = 1 + 5 = 6 < 8 \quad i++$$

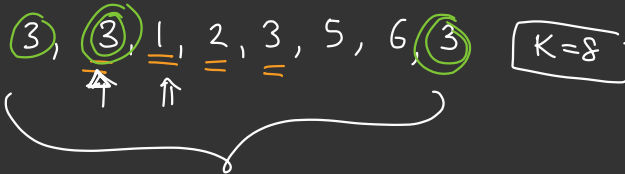
$$= 2 + 5 = 7 < 8 \quad i++$$

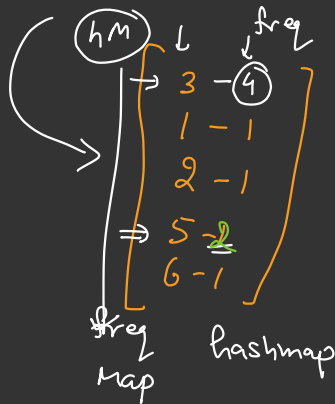
$$= 3 + 5 = 8 = 8 \quad i++, j--$$

using HashSet

$O(N)$ time
but $O(N)$ space

What if
Duplicates





(3, 5)
(3, 5)

$$3 + 4 = 8$$

$$3 + 4 = 6$$

$$\Rightarrow y = 3$$

$$K = 6$$

$$x = y = 3$$

freq[x]₂

x = y

$$4C_2 = \frac{4 \times 3}{2} = 6 \text{ ways}$$

$$2C_1 \cdot 4C_1$$

$$= 2 \times 4 = 8 \text{ ways}$$

(x, y) freq[x] freq[y]

3 3 3 5 5 3

3 → 4
1 → 1
2 → 1
5 → 2
6 → 1

3 → 5?

2 → 6

5 → 3

6 → 2

$$4 \times 2 = 8$$

$$1 \times 1 = 1$$

X DNC

X DNC

in good.

$$a(i) < a(j)$$

(5, 3)

$$a(j) > a(i)$$

① Step 1 itr over array & update hashmap

for (x : arr) {

hm[x] += 1

}

ans = 0

② for (key : hashmap) {

x = key

T = 6

y = T - x

3, 3, 3, 3

if (x == y) {

ans += hm[x] C₂ ;

3 → ④

}

else if (x < y && y is in hashmap) {

ans += (hm[x] * hm[y]),

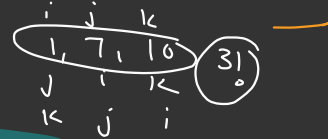
3

{

Triplet SumUnsorted

Array = [1, 3, 5, 7, 2, 6, ..., 10, 11]

$$a[i] + a[j] + a[k] = T$$

 $i < j < k$ Brute Force

```

for (i=0, i <= n-3; i++) {
  for (j=i+1, j <= n-2; j++) {
    for (k=j+1, k <= n-1; k++) {
      if (cond == T) {
        //
      }
    }
  }
}

```

~~for (i=0, i <= n-3; i++) {
 for (j=i+1, j <= n-2; j++) {
 for (k=j+1, k <= n-1; k++) {~~

↓
 Duplicate
 Triplets

3-Sum

Two Sum
→ [1, 3, 5, 7, 2, 6, 10, -----]
⇒ Sort(-) ↑ ↑ ⇐ ↑
 i j k

T = [9]
↑

for(i=0 ----- n-3) {

current = a(i)

T' = T - a(i)

Pair of No's whose sum is T'

$O(N \log N)$

+ $O(N^2)$

= $O(N^2)$

Hashing

N * N
= $O(N^2)$

Sorting

+
2 Pointer

= $O(N \log N)$

+ $O(N^2)$

= $O(N^2)$

Two
Sum

Two pointer

(good
choice)

j = i + 1

k = n - 1

while (j < k) {

 if (a(j) + a(k) == T') { ----- }

 < T' { j++ }

 > T' { k-- }

}

}

Google Interview (2020)

1, 3, 5, 7, 2, 11, 0 ← Input

(No duplicates)

Count the no of triplets s.t

$$a[i] + a[j] + a[k] \leq T$$

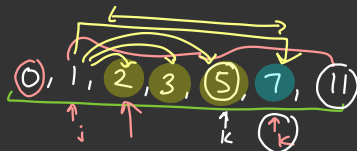
$$T = 8$$

9.55

7 mins.

Medium

Sort



$$1 + 11 > 8$$

for (i=0, i<=n-3, i++) {

$$T' = T - arr[i]$$

$$j = i + 1$$

$$k = n - 1$$

while (j <= k) {

> → if (a[j] + a[k] > T') {
 k-- ;

≤

else {

$$\text{count} = \text{count} + \underline{(k - j)} ;$$

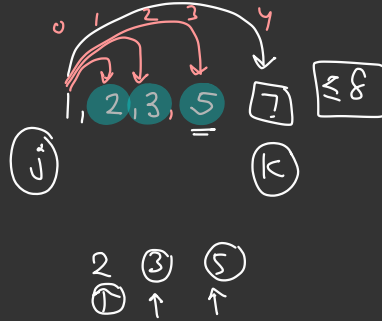
$$\underline{j++} ;$$

}

optimisation
 $\left\{ \begin{array}{l} 0, 1, 7 \\ 0, 1, 5 \\ 0, 1, 3 \\ 0, 1, 2 \end{array} \right\}$

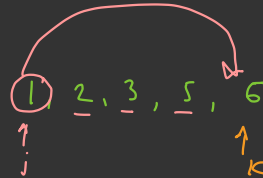
Dry Run

{
i=0



2 3 5
↑ ↑ ↑

Problem



Update k

j++;

a	b	c
<u>0</u>	<u>1</u>	<u>11</u>
0	1	7
0	2	7
0	2	5
0	3	5

~~1+11 ≤ 8~~ k--

{
1+7 ≤ 8
1+5 ≤ 8
1+3 ≤ 8
1+2 ≤ 8
}

~~2+7 ≤ 8~~ k--

{
2+5 ≤ 8
2+3 ≤ 8
3+5 ≤ 8
}

1+6 ≤ 8

2,6 → x

1+6 ≤ 8

Contribution → 4

Q

N sorted elements, check if

pair $a(i), a(j)$ st

$$\underline{a[i]} - \underline{a[j]} = k$$

$$k > 0$$

Sample -

$[-3, 0, 1, 3, 6, 8, 11, 14, 18, 25]$
↑ ↑ ↑

$$k = 5$$

① Brute Force $O(N^2)$

② Binary Search $O(N \log N)$

for (x : arr) {
 BS for (x+k)
}

$$x - y = k$$
$$\Rightarrow x = k + y$$

-3, ② ← BS
0, ⑤ ← BS
1, ⑥ ← BS

④, ⑨
↑ ↑

③ Hashing $O(N)$ with $O(N)$ Space
[x,
 look for x+k in hashtable

④ Two Pointer

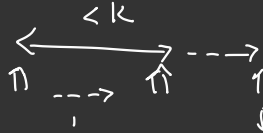
$[-3, 0, 1, 3, 6, 8, 11, 14, 18, 25]$

Problem

can't
init
at ends

$25 - (-3) = 28$ (Big)
 $18 - (-3) = 21$ (j--)
 $25 - 0 = 25$ (i++)

Diff = 5



$0 - (-3) = 3 < 5 \quad j++ \quad \text{Diff} = 5$
 $1 - (-3) = 4 < 5 \quad j++$
 $3 - (-3) = 6 > 5 \quad i++$
 $3 - 0 = 3 < 5 \quad j++$
 $6 - 0 = 6 > 5 \quad i++$
 $6 - 1 = 5 = 5 \quad \text{found}$

①
or
②



At max
2N iterations

bool checkDiffK (A , N)

i = 0

j = 1

while (j < N) {

if ($a_j - a_i == k$) {

return True,

}

else if ($a_j - a_i > k$) {

i++ // Contraction

if (i == j) { j++ }

}

else {

j++ // Expansion

}

}

k = 2

[6 | 9 | 10 | 12]

↑ ↑ ↑ ↑
i j
9 - 6 > 2

Q

Arr 1 = [1, 3, 2, 5, 7, 8]

Arr 2 = [3, 7, 8]
 ↑ ↑ ↑

Yes

Arr = [2, 1, 8]

No

Arr = [3, 15, 7]

No

Subsequence with
(elements will have
same relative
order)

↓
Yes

bool check (Arr 1 , Arr 2) {

$$O(N + M) = O(N)$$

A1 $\Rightarrow$ 1, 2, 3, 5, 7, 6, 9, 20, 8, 15, 10, 14, 21 $i = n-1$
 $j = m-1$

A2 $\Rightarrow$ 5, 7, 6, 8, 10
 1.
 j

time
 $O(N)$
 $O(1)$
 space

3 5 7 8
 $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
 $i = -1$ 3 11 8
 $\uparrow$ $\uparrow$
 j j

$j = 0$
 $\downarrow$
 while ($i \geq 0$ & $j > 0$) {
 $a1[i] = a2[j]$
 $i--$
 $a1[i] == a2[j]$
 $i--$
 $\Rightarrow$ $j--$
 $\Rightarrow$ if ($j == -1$) {
 $\quad$ return True;
 $\quad$ }
 }

-
return false,

}